TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA

UTN
UNIVERSIDAD TECNOLÓGICA NACIONAL

Trabajo Práctico Integrador

Metodología de Sistemas II

Integrantes:

- Grela, Ariel
- Higa, Matías

Comisión N° 1

M25 C4-06

Fecha:

Tabla de Contenidos

Tabla de Contenidos **2**
Resumen Ejecutivo **3**
Etapas **3**
Etapa 1: Selección y descripción del sistema base 3
Etapa 2: Calidad externa vs. Calidad interna 7
2.1 Sobre la calidad 7
2.2 Calidad externa 8
2.2.1 Verificación funcional 8
2.2.2 Experiencia de usuario 10
2.3 Calidad interna 11
Etapa 3: Deuda técnica y code smells 14
CS-01 - Long Method 14
CS-02 - Large Class 14
CS-03 - Lógica de negocio acoplada a la consola 15
CS-04 - Manejo de errores inconsistente 15
CS-05 - Duplicated Code 15
CS-06 - Shotgun Surgery 16
CS-07 - Primitive Obsession 16
CS-08 - Switch Statements 16
CS-09 - Inappropriate Intimacy (Encapsulamiento roto) 17
CS-10 - Data Class 17
CS-11 - Nombres poco descriptivos 17
CS-12 - Persistencia y datos semilla mezclados con el dominio 17
CS-13 - Long Parameter List 18
Etapa 4: Aplicacion de SOLID y Clean Code 18
Etapa 5: Refactoring seguro 18
Etapa 6: Pruebas y verificación del comportamiento 18
Etapa 7: Code Review profesional 18
Etapa 8: Conclusiones y aprendizaje 18
Conclusiones y mejoras futuras **18**
Anexo y Enlaces **19**
Anexos 19
Repositorio GitHub 19
Video 19

Resumen Ejecutivo

Breve síntesis del sistema analizado, problema principal, mejoras aplicadas y conclusión técnica.

A completar al finalizar todo.

Etapas

Etapa 1: Selección y descripción del sistema base

Dominio

Sistema de Gestión de Solicitudes Académicas. Permite registrar, clasificar y dar seguimiento a los pedidos que realizan los estudiantes, como constancias, pedidos de equivalencias, reclamos, y otros. Su objetivo es centralizar estas solicitudes, conocer en qué estado se encuentra cada una y saber quién es responsable de resolverla.

Alcance

El alcance de módulo es deliberadamente reducido. Como plantea la consigna, la complejidad del trabajo no está en la cantidad de funcionalidad, sino en el diagnóstico de calidad y en la mejora de la estructura interna del código

El sistema comprende el ciclo de vida de una solicitud, desde su registro hasta su resolución, junto con la consulta y un reporte básico. Quedan fuera del alcance la autenticación de usuarios, la gestión de estudiantes (alta, baja, modificación), interfaz gráfica, las notificaciones por correo y la integración con otros sistemas institucionales.

Actores

- **Estudiante:** Realiza solicitudes y consulta su estado.
- **Responsable:** Personal de la institución educativa. Se encarga de asignar y reasignar las solicitudes, gestionar las que tiene a su cargo y actualizar su estado. También genera los reportes del sistema.

Entidades principales

Entidad	Descripción
Solicitud	Pedido realizado por un estudiante. Tiene tipo, descripción, prioridad, estado, fecha y responsable asignado.
Estudiante	Persona que origina la solicitud (legajo, nombre, carrera)
Responsable	Persona o sector encargado de resolver la solicitud
Estado	Situación de la solicitud: Pendiente / En proceso / Resuelta / Rechazada

Funcionalidades

1. **Registrar solicitud:** Alta de una nueva solicitud asociada a un estudiante, con tipo y prioridad.
2. **Cambiar estado:** Actualización del estado de una solicitud existente.
3. **Asignar responsable:** Vinculación de una solicitud con un responsable o área.
4. **Buscar / filtrar solicitudes:** Consulta por estado, tipo, prioridad o estudiante.
5. **Listar reporte:** Resumen de solicitudes agrupadas por estado o prioridad.

Tecnología utilizada

- **Lenguaje:** Python 3.12 / Java 21
- **Persistencia:** Estructuras en memoria
- **Interfaz:** Consola
- **Control de versiones:** Git y GitHub
- **Entorno de desarrollo:** IntelliJ Idea

Versión inicial

La versión inicial del sistema es perfectamente funcional desde el punto de vista del usuario: Permite realizar todas las operaciones descritas y produce los resultados esperados. Sin embargo, fue construida sin atender a criterios de diseño y presenta problemas internos que se analizan en las etapas siguientes. Entre ellos se encuentran la concentración de responsabilidades en pocas clases o funciones, la duplicación de código, los nombres poco descriptivos y el acoplamiento directo entre la lógica de negocio y la persistencia.

Esta versión quedó identificada en el repositorio con el tag **estado-inicial**, que sirve como punto de comparación para evaluar las mejoras (sistema base).

Funcionamiento del sistema:

★ **Menú principal**

```
21:16:40: Executing ':tupad.ms2.Main.main()'…

> Task :compileJava UP-TO-DATE
> Task :processResources NO-SOURCE
> Task :classes UP-TO-DATE

> Task :tupad.ms2.Main.main()

######################## GESTION DE SOLICITUDES ACADEMICAS ########################
1. Registrar solicitud
2. Cambiar estado
3. Asignar responsable
4. Buscar / filtrar
5. Reporte
0. Salir
Opcion:
```

★ **Funcionalidad: Registro de solicitud (opción 1 en menú)**

1) Selección de estudiante y tipo

```
Estudiante:
1. L1001 - Agustin Di Maria (agustin87@gmail.com)
2. L1002 - Lucia Kempes (lucia.kempes@outlook.com)
3. L1003 - Tomas Maradona (tomas.mdz@hotmail.com)
4. L1004 - Valentina Passarella (vale.passarella@yahoo.com)
5. L1005 - Nicolas Fillol (nfillol23@gmail.com)
6. L1006 - Camila Batistuta (camila.batistuta@outlook.com)
7. L1007 - Franco Houseman (fr.houseman@gmail.com)
8. L1008 - Martina Ardiles (martina.ardiles@hotmail.com)
9. L1009 - Santiago Bertoni (sbert0ni@yahoo.com)
10. L1010 - Julieta Valdano (julieta.valdano@gmail.com)
11. L1011 - Mateo Ruggeri (mateo.ruggeri@outlook.com)
12. L1012 - Florencia Gallego (flor.gallego@hotmail.com)
13. L1013 - Benjamin Tarantini (benja.tarantini@gmail.com)
14. L1014 - Sofia Houseman (sofia.h77@yahoo.com)
15. L1015 - Facundo Rattin (facundo.rattin@outlook.com)
16. L1016 - Micaela Sivori (mica.sivori@gmail.com)
17. L1017 - Ignacio Labruna (ignacio.labruna@hotmail.com)
18. L1018 - Paula Ayala (paula.ayala@yahoo.com)
19. L1019 - Agustin Caniggia (agus.caniggia@gmail.com)
20. L1020 - Julian Brown (julian.brown@outlook.com)
Opcion: 6
Tipo:
1. CONSTANCIA
2. EQUIVALENCIA
3. RECLAMO
4. OTRO
Opcion: 1
```

2) Selección de tipo -> descripción de la solicitud -> confirmación de solicitud registrada.

```
Tipo:
1. CONSTANCIA
2. EQUIVALENCIA
3. RECLAMO
4. OTRO
Opcion: 1
Descripcion: Constancia de alumno regular
Prioridad (1=alta, 2=media, 3=baja): 2
Solicitud registrada con id 4
```

★ **Funcionalidad: Cambio de estado (opción 2 en menú)**

Selección de solicitud -> cambio de estado

```
--- Solicitudes ---
#1 | Agustin Di Maria | CONSTANCIA | Prioridad 2 | PENDIENTE | SIN ASIGNAR
#2 | Lucia Kempes | RECLAMO | Prioridad 1 | EN_PROCESO | Maria Ortega S◆nchez (Alumnado)
#3 | Agustin Di Maria | EQUIVALENCIA | Prioridad 3 | PENDIENTE | SIN ASIGNAR
#4 | Camila Batistuta | CONSTANCIA | Prioridad 2 | PENDIENTE | SIN ASIGNAR
Id solicitud: 2
Nuevo estado:
1. EN_PROCESO
2. RESUELTA
3. RECHAZADA
Opcion: 3
Estado actualizado a RECHAZADA
```

★ **Funcionalidad: Asignar responsable (opción 3 en menú)**

Selección de solicitud -> Asignar responsable

```
--- Solicitudes ---
#1 | Agustin Di Maria | CONSTANCIA | Prioridad 2 | PENDIENTE | SIN ASIGNAR
#2 | Lucia Kempes | RECLAMO | Prioridad 1 | EN_PROCESO | Maria Ortega S◆nchez (Alumnado)
#3 | Agustin Di Maria | EQUIVALENCIA | Prioridad 3 | PENDIENTE | SIN ASIGNAR
Id solicitud: 3
Responsable:
1. Maria Ortega S◆nchez (Alumnado)
2. Jorge Tchami (Alumnado)
3. Laura Gorosito (Secretaria Academica)
Opcion: 2
Responsable asignado: Jorge Tchami
```

★ **Funcionalidad: Buscar / filtrar (opción 4 en menú)**

Ejemplo buscando x tipo de solicitud: Seleccionar buscar por tipo (2) -> Seleccionar tipo de solicitud

```
Buscar por:
1. Estado
2. Tipo
3. Legajo
4. Prioridad
Opcion: 2
Tipo:
1. CONSTANCIA
2. EQUIVALENCIA
3. RECLAMO
4. OTRO
Opcion: 1
#1 | Agustin Di Maria | CONSTANCIA | Prioridad: 2 | PENDIENTE
Resultados: 1
```

★ **Funcionalidad: Reporte (opción 5 en menú)**

```
===== REPORTE DE SOLICITUDES =====
#1 | 2026-10-21 | Agustin Di Maria | CONSTANCIA | Prioridad: 2 | PENDIENTE | SIN ASIGNAR
#2 | 2026-10-21 | Lucia Kempes | RECLAMO | Prioridad: 1 | EN_PROCESO | Maria Ortega S◆nchez (Alumnado)
#3 | 2026-10-21 | Agustin Di Maria | EQUIVALENCIA | Prioridad: 3 | PENDIENTE | Jorge Tchami (Alumnado)
---------------------------------
Total: 3
Pendientes: 2 | En proceso: 1 | Resueltas: 0 | Rechazadas: 0
Alta: 1 | Media: 1 | Baja: 1
```

Etapa 2: Calidad externa vs. Calidad interna

2.1 Sobre la calidad

La calidad de un sistema puede analizarse desde dos miradas diferentes:

- La **calidad externa** es la que percibe el usuario al utilizar el sistema: Que las funcionalidades hagan lo que prometen, que los resultados sean correctos, datos inválidos rechazados, que el sistema “funcione bien” (no falle y sea rápido).
- En cambio, la **calidad interna**, es la que percibe quien debe leer, mantener o modificar el código: La claridad de los nombres, las responsabilidades, la ausencia de duplicación, el grado de acoplamiento entre componentes, la facilidad para probarlo y mantenerlo.

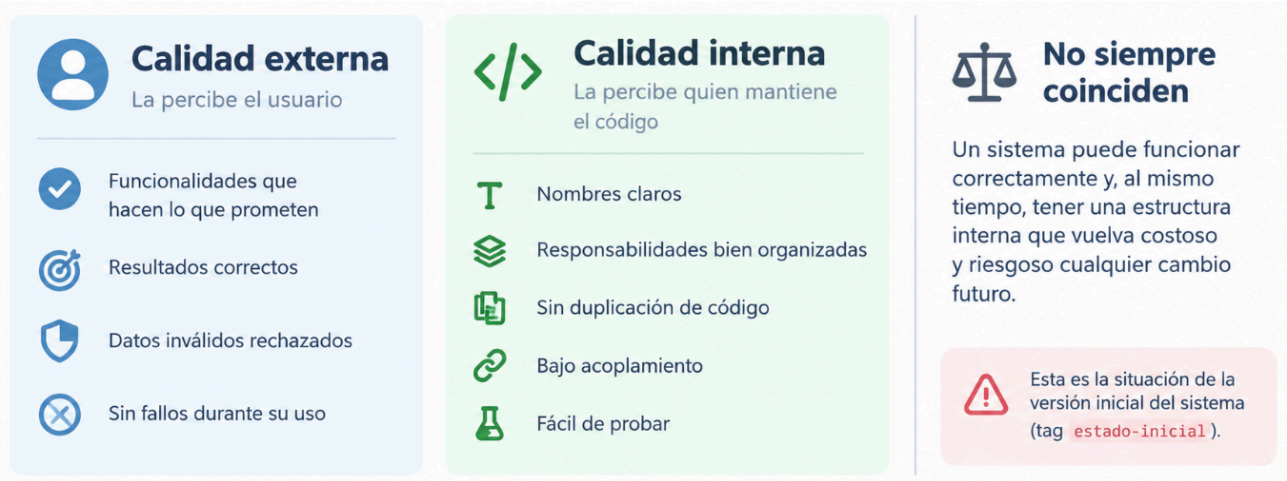


Fig. imagen CA-03

2.2 Calidad externa

Desde la perspectiva del usuario, la versión **estado-inicial** cumple con todas las funcionalidades que este actor puede percibir.

2.2.1 Verificación funcional

Se ejecutó el sistema por consola, probando así cada operación con datos válidos como con datos inválidos. El sistema inicia con 3 solicitudes precargadas:

1. Solicitud de constancia (sin asignar, sin estado)
2. Solicitud de reclamo (con responsable asignado, estado “en proceso”)
3. Solicitud de equivalencia (sin asignar, sin estado)

Funcionalidad	Caso	Resultado	ID Evidencia (ver Anexo)
Registrar solicitud	Alta con todo válido (estudiante, tipo, descripción, prioridad)	“Solicitud registrada con id 4”	CA-05
Registrar solicitud	Descripción con menos de 5 caracteres	“Error: descripcion muy corta” y no se registra	CA-06
Registrar solicitud	Prioridad fuera de rango	“Error: prioridad invalida” y no se registra	CA-07
Cambiar estado	Cambiar estado sin responsable asignado	El sistema lo informa y solo ofrece la opción RECHAZADA	CA-08
Cambiar estado	Cambiar estado: EN_PROCESO → RESUELTA	“Estado actualizado a RESUELTA”	CA-09
Cambiar estado	Intentar modificar una solicitud cerrada	“Error: la solicitud esta cerrada”	CA-10
Asignar responsable	Reasignar una solicitud que ya tiene un responsable	Muestra el responsable actual, avisa que se reemplaza y asigna el nuevo	CA-11
Buscar / filtrar	Búsqueda x estado EN_PROCESO	Lista la solicitud #2 – “Resultados: 1”	CA-12
Buscar / filtrar	Búsqueda por legajo L1001	Lista las solicitudes #1 y #3 – “Resultados: 2”	CA-13
Listar reporte	Reporte con datos precargados	Total: 3. Pendientes: 2, En proceso: 1. Alta: 1, Media: 1, Baja: 1	CA-14
Menú	c	“Opcion invalida” y vuelve al menú	CA-15

Como se observa, el sistema respeta las reglas del dominio: valida los datos de entrada, controla las transiciones de estado (no permite pasar a “En proceso” sin un responsable ni modificar solicitudes cerradas) y permite cancelar las selecciones desde el menú. Desde el punto de vista del usuario, la calidad externa es buena.

Limitación detectada: Durante las pruebas se encontró un defecto visible. Al registrar una solicitud, si en el campo prioridad se ingresa un valor no numérico (por ejemplo “ALTA”), el sistema lanza una **NumberFormatException** no controlada y el programa finaliza abruptamente:

```
Execution failed for task ':tupad.ms2.Main.main()'.
> Process 'command 'C:\Users\matut\.jdks\ms-21.0.12.1\bin\java.exe'' finished with non-zero exit value 1

* Try:
> Run with --stacktrace option to get the stack trace.
> Run with --info or --debug option to get more log output.
> Run with --scan to get full insights from a Build Scan (powered by Develocity).
> Get more help at https://help.gradle.org.
BUILD FAILED in 17m 39s
2 actionable tasks: 1 executed, 1 up-to-date
```

Fig. imagen CA-16

Como la persistencia es en memoria, se pierden todas las solicitudes cargadas durante la sesión. El origen es una lectura no protegida, en **Main.java**, línea 58:

```
50                  String[] tipos = {"CONSTANCIA", "EQUIVALENCIA", "RECLAMO", "OTRO"};
51                  int t = elegir(sc, titulo: "Tipo:", tipos);
52                  if (t == -1) {
53                      System.out.println("Registro cancelado.");
54                  } else {
55                      System.out.print("Descripcion: ");
56                      String d = sc.nextLine();
57                      System.out.print("Prioridad (1=alta, 2=media, 3=baja): ");
58                      int p = Integer.parseInt(sc.nextLine());
59                      g.registrarSolicitud(est.legajo, tipos[t], d, p);
60                  }
61              }
62          }
```

Fig. imagen CA-17

El caso muestra que la calidad externa es buena pero no completa, y que una falla interna (manejo de errores inconsistente) termina teniendo un impacto directo en el usuario.

2.2.2 Experiencia de usuario

Para complementar la verificación funcional, se evaluó la calidad externa del sistema desde la perspectiva de la experiencia de usuario. Se usó una prueba de usabilidad con tareas guiadas.

> **Nota:** Los valores de la prueba con usuarios son datos simulados. Se construyeron a partir de los comportamientos reales observados en la versión **estado-inicial** y no provienen de una prueba con participantes reales.

Metodología de la prueba con usuarios

- **Participantes:** 5 (P1 a P5), estudiantes de la tecnicatura sin conocimiento previo del sistema.
- **Modalidad:** ejecución por consola desde IntelliJ, con datos precargados. No se dio ayuda durante las tareas.
- **Éxito:** se considera exitosa la tarea completada en el primer intento, sin ayuda externa.
- **Tiempo:** desde que el participante lee la consigna hasta que obtiene el resultado esperado.
- **Error:** cualquier entrada que produzca un mensaje de error, una cancelación o el cierre del programa.

Tarea	Consigna dada al participante
T1	Registrar una solicitud de constancia de alta prioridad para el estudiante L1005
T2	Poner en proceso la solicitud #1
T3	Asignar a Laura Gorosito como responsable de la solicitud #3
T4	Buscar todas las solicitudes del estudiante L1001
T5	Generar el reporte general de solicitudes

Resultados por tarea (datos simulados)

Tarea	Tasa de éxito	Tiempo promedio	Errores totales	Observación principal
T1 – Registrar solicitud	3/5 (60 %)	58 s	4	P2 escribió “alta” en la prioridad y el programa se cerró. P4 ingresó una descripción corta y tuvo que repetir todo el registro.
T2 – Poner en proceso	2/5 (40 %)	71 s	5	Tres participantes no sabían que primero había que asignar un responsable. El sistema lo informa, pero obliga a salir y entrar a otra opción.
T3 – Asignar responsable	5/5 (100 %)	24 s	0	Sin dificultades.
T4 – Buscar por estudiante	4/5 (80 %)	21 s	1	En la lista de 20 estudiantes, P5 eligió un número equivocado.

T5 – Generar reporte	5/5 (100 %)	6 s	0	Sin dificultades.
Total	**19/25 (76 %)**	**36 s**	**10**	—

Satisfacción percibida – SUS (datos simulados)

Al terminar, cada participante completó el cuestionario SUS (*System Usability Scale*, 10 preguntas, puntaje de 0 a 100).

Participante	P1	P2	P3	P4	P5	Promedio
Puntaje SUS	72,5	65,0	77,5	60,0	75,0	70,0

2.3 Calidad interna

A continuación, se agrupan los problemas según el atributo de calidad afectado. El detalle en profundidad de cada uno, con su prioridad de corrección, se desarrolla en la Etapa 3.

Legibilidad

- Muchos nombres son abreviaturas que no comunican su propósito: **g** (gestor), **sc** (scanner), **op**, **x**, **r**, **p**, **d**, y en las entidades **est**, **resp** y **desc**.
- Números mágicos: La prioridad se representa con los enteros 1, 2 y 3, y su significado solo queda documentado en un comentario.
- El tipo de solicitud es un **String**, tampoco tiene una representación propia.

```
4
5   public class Solicitud {  23 usages  Matias H
6       public int id;  12 usages
7       public Estudiante est;  5 usages
8       public Responsable resp;  12 usages
9       public String tipo;        // CONSTANCIA, EQUIVALENCIA, RECLAMO, OTRO  5 usages
10      public int prioridad;      // 1 = alta, 2 = media, 3 = baja  8 usages
11      public Estado estado;  23 usages
12      public String desc;  1 usage
13      public String fecha;  2 usages
14
```

Fig. imagen CA-18

Duplicación

- La lista de tipos válidos (**"CONSTANCIA"**, **"EQUIVALENCIA"**, **"RECLAMO"**, **"OTRO"**) aparece multiples veces (por ej, en Main.java aparece en las líneas 50 y 137)
- El recorrido para buscar una solicitud por id está escrito tres veces (**cambiarEstado()**, **asignarResponsable()** y **Main.pedirSolicitud()**)
- El formateo **nombre + " (" + area + ")"** se arma en 4 lugares distintos.

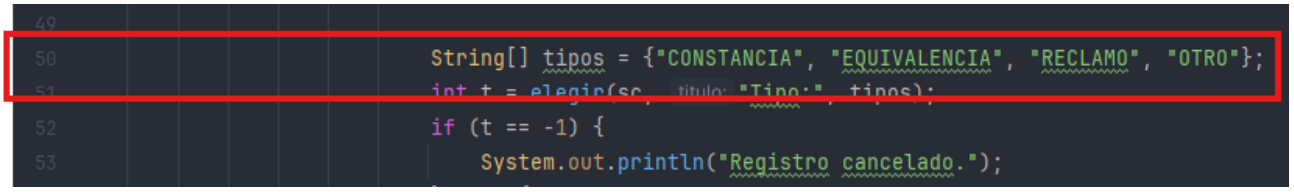


Fig. imagen CA-19

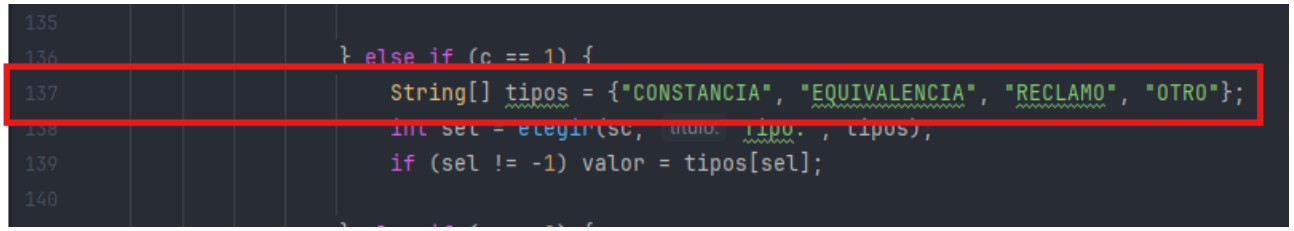


Fig. imagen CA-20

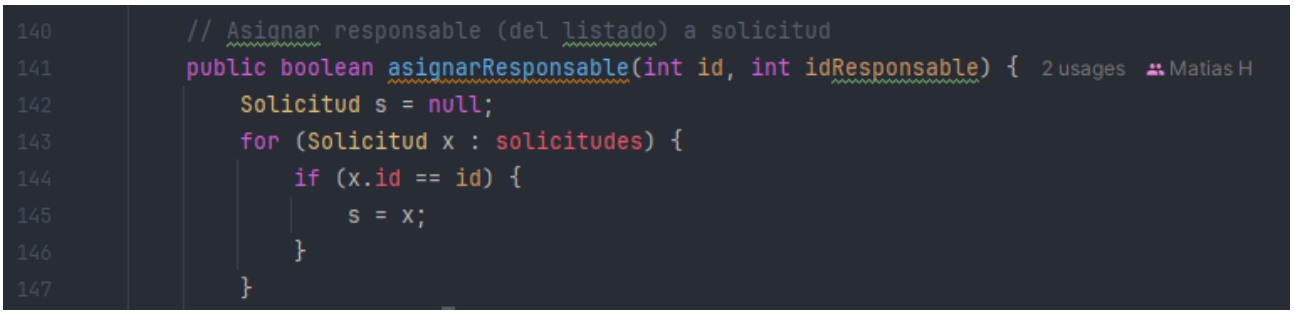


Fig. imagen CA-21

Cohesión

Las responsabilidades están concentradas. Una misma clase tiene múltiples motivos para cambiar:

- main() tiene más de 150 líneas y tiene múltiples responsabilidades (presentación del menú, lectura de datos, validaciones).
- GestorSolicitudes carga los datos de inicio, aplica las reglas de negocio, búsquedas, arma el reporte y lo imprime por consola.

Acoplamiento

La lógica de negocio está acoplada a la consola: los métodos del gestor informan sus resultados y errores mediante System.out.println. La consecuencia más clara aparece en **cargarDatosEjemplo()**, que para precargar datos sin mostrar mensajes debe redirigir temporalmente la salida estándar:

```
178     // Precarga de solicitudes al iniciar, sin mostrar los mensajes del gestor (estudiantes y responsables ya vienen cargados desde GestorSolicitudes)
179 @   private static void cargarDatosEjemplo(GestorSolicitudes g) {  1 usage   Matias H *
180         PrintStream original = System.out;
181         System.setOut(new PrintStream(OutputStream.nullOutputStream()));
182         try {
183             g.registrarSolicitud( legajo: "L1001",  tipo: "CONSTANCIA",  desc: "Constancia de alumno regular",  prioridad: 2);
184             g.registrarSolicitud( legajo: "L1002",  tipo: "RECLAMO",  desc: "Error en nota del parcial",  prioridad: 1);
```

Fig. imagen CA-22

Manejo de errores y testeabilidad

Los errores se comunican de forma inconsistente: algunos métodos devuelven **-1**, otros **false**, y todos imprimen un mensaje. Actualmente el sistema no cuenta con pruebas.

Conclusión de la etapa

La versión inicial demuestra que **“funciona” no es lo mismo que “se puede mantener”**. Desde la mirada del usuario el sistema cumple sus funcionalidades y respeta las reglas del dominio, con una única falla visible en la validación de la prioridad. Desde la mirada del desarrollador, la estructura interna presenta duplicación, baja cohesión, alto acoplamiento con la consola y un manejo de errores inconsistente. Implementar un cambio que conceptualmente es pequeño obliga a tocar varios archivos y requiere conocer en detalle dónde está repetida cada regla.

Etapa 3: Deuda técnica y code smells

Se revisó el código de **estado-inicial** buscando code smells y debilidades técnicas generales, con el fin de detectar problemas de diseño que eleven el costo del cambio.

Clasificacion de prioridades de corrección	
Alta	Causa defectos visibles, bloquea otras mejoras (por ejemplo las pruebas) o concentra la mayor parte de los cambios futuros
Media	Dificulta el mantenimiento pero no rompe nada por sí solo
Baja	Afecta la prolijidad y su corrección es barata y sin riesgo

CS-01 - Long Method

Evidencia:	CAP-23 en Anexo Método muy largo (más de 170 líneas), con un gran bloque de condicionales anidados
Ubicación:	Main.main()
Impacto:	Las opciones se puede probar por separado
Riesgo:	Fácil de romper algo accidentalmente, ya que todo cambio pasa por este método
Prioridad:	Alta. Es conveniente separar las tareas en distintos métodos (Extract Method)

CS-02 - Large Class

Evidencia:	CAP-24 en Anexo Clase con muchas responsabilidades (precarga de datos, guarda listas, validaciones, búsqueda, cálculos)
Ubicación:	GestorSolicitudes
Impacto:	Viola SRP (más de una responsabilidad)
Riesgo:	Cualquier modificación puede afectar funcionalidades que no tienen relación con ella
Prioridad:	Alta. Es conveniente separar las responsabilidades en distintas clases

CS-03 - Lógica de negocio acoplada a la consola

Evidencia:	CAP-25 en Anexo GestorSolicitudes usa salida por consola (System.out) para devolver respuesta y errores
Ubicación:	GestorSolicitudes, Main.cargarDatosEjemplo()
Impacto:	Viola SRP en GestorSolicitudes. Muestra la info directo por texto en consola, en lugar de devolver un resultado/excepción.
Riesgo:	Al mostrar los mensajes por consola, no se pueden reutilizar. Al momento de la ejecución, se tuvo que “silenciar” los mensajes temporalmente para que no se muestren los de la precarga de datos.
Prioridad:	Alta. Bloquea las pruebas.

CS-04 - Manejo de errores inconsistente

Evidencia:	Métodos informan el error de una distinta manera: ● registrarSolicitud() devuelve -1 (CAP-26) ● cambiarEstado() devuelve false (CAP-27) ● asignarResponsable() devuelven false (CAP-28) ● buscar() imprime un mensaje (CAP-29) Catch vacíos (CAP-29) pedirSolicitud() devuelve null (CAP-30)
Ubicación:	Main, GestorSolicitudes
Impacto:	No se puede saber por qué falló una operación
Riesgo:	Los catch vacios no aportan info sobre las fallas. Un error puede terminar el programa o informarse con un mensaje equivocado.
Prioridad:	Alta

CS-05 - Duplicated Code

Evidencia:	CAP-31, CAP-32, CAP-33, CAP-34
Ubicación:	Main, GestorSolicitudes
Impacto:	Un cambio único obliga a buscar y editar todas las copias
Riesgo:	Dificil de mantener.
Prioridad:	Alta

CS-06 - Shotgun Surgery

Evidencia:	CAP-35, CAP-36, CAP-37, CAP-38 Agregar un nuevo tipo de solicitud requeriría cambiar el código en distintas partes del proyecto
Ubicación:	Main, GestorSolicitudes, Solicitud
Impacto:	Una modificación requiere hacer cambios en varios lugares distintos
Riesgo:	Omitir/olvidarse de cambiar en un lugar hace el sistema inconsistente.
Prioridad:	Alta

CS-07 - Primitive Obsession

Evidencia:	CAP-39, CAP-40, CAP-41, CAP-42, CAP-43 Primitivos en lugar de objetos
Ubicación:	Main, GestorSolicitudes, Solicitud
Impacto:	Validación manual
Riesgo:	Un error de tipeo (ej. “RESUELTHO ”) compila, falla en ejecución
Prioridad:	Media

CS-08 - Switch Statements

Evidencia:	CAP-44, CAP-45, CAP-46 Switch complejos, If anidados
Ubicación:	Main, GestorSolicitudes
Impacto:	Viola OCP. Agregar un criterio o un estado obliga a modificar código que ya funciona.
Riesgo:	Agregar más ramas puede romper las existentes.
Prioridad:	Media

CS-09 - Inappropriate Intimacy (Encapsulamiento roto)

Evidencia:	CAP-47, CAP-48, CAP-49 Atributos públicos (Solicicitud, Estudiante, Responsable), listas públicas en GestorSolicitudes, Main lee g.solicitudes
Ubicación:	Main, GestorSolicitudes, Solicitud, Estudiante, Responsable
Impacto:	El sistema permite modificar datos importantes sin pasar por las validaciones del negocio
Riesgo:	Cualquier programador puede modificar estos datos
Prioridad:	Media

CS-10 - Data Class

Evidencia:	CAP-50, CAP-51 Solicitud no tiene reglas. Sus reglas están en GestorSolicitudes y Main.
Ubicación:	Main, GestorSolicitudes, Solicitud
Impacto:	Main y el gestor deciden por la solicitud
Riesgo:	La interfaz y el dominio pueden contradecirse
Prioridad:	Media

CS-11 - Nombres poco descriptivos

Evidencia:	CAP-52 ej. p, c y sEst, est, resp
Ubicación:	Main, GestorSolicitudes
Impacto:	Es necesario leer el contexto para descrifrar la variable
Riesgo:	Los nombres poco descriptivos hacen que sea más fácil equivocarse de variable al modificar.
Prioridad:	Media

CS-12 - Persistencia y datos semilla mezclados con el dominio

Evidencia:	CAP-53, CAP-54 Las listas en memoria funcionan como persistencia y están dentro del gestor. La precarga de solicitudes está en main
Ubicación:	Main, GestorSolicitudes
Impacto:	Viola DIP, el gestor depende de un detalle concreto. Pasar a archivo o a base de datos obliga a reescribir GestorSolicitudes.
Riesgo:	Sin gestor vacío para pruebas
Prioridad:	Media

CS-13 - Long Parameter List

Evidencia:	CAP-55 El constructor de `Solicitud` recibe 6 parámetros
Ubicación:	Solicitud, GestorSolicitudes
Impacto:	Viola DIP, el gestor depende de un detalle concreto. Pasar a archivo o a base de datos obliga a reescribir GestorSolicitudes.
Riesgo:	Sin gestor vacío para pruebas
Prioridad:	Media

Etapa 4: Aplicacion de SOLID y Clean Code

pendiente

Etapa 5: Refactoring seguro

pendiente

Etapa 6: Pruebas y verificación del comportamiento

pendiente

Etapa 7: Code Review profesional

pendiente

Etapa 8: Conclusiones y aprendizaje

pendiente

Conclusiones y mejoras futuras

pendiente

Anexo y Enlaces

Anexos

Anexo en documento adjunto.

Repositorio GitHub

Enlace al repositorio GitHub: **https://github.com/emegiga/MS2_TPI_Grela_Higa**

Video

LINK